

## Quiz: Indexes (1)

### Collaboration policy:

You may NOT:

1. discuss the exam with any human other than Mike; this includes:
  - (a) asking your friend for clarification about what a problem is asking
  - (b) asking your friend if they've finished working on the quiz
  - (c) posting questions to github, stackoverflow, or any other online forum

You may:

1. take as much time as needed
2. use any written notes / electronic resources you would like (including postgres documentation and AI systems like ChatGPT)
3. use the lambda server and connect to a running postgres instance
4. ask Mike to clarify questions via email

Name:

---

All of the problems assume the following schema.

```
CREATE TABLE users (  
    id_users BIGINT PRIMARY KEY,  
    created_at TIMESTAMPTZ,  
    username TEXT UNIQUE NOT NULL  
);  
  
CREATE TABLE tweets (  
    id_tweets BIGINT UNIQUE NOT NULL,  
    id_users BIGINT REFERENCES users(id_users),  
    in_reply_to_user_id BIGINT REFERENCES users(id_users),  
    created_at TIMESTAMPTZ,  
    text TEXT  
);  
  
CREATE TABLE tweet_tags (  
    id_tweets BIGINT REFERENCES tweets(id_tweets),  
    tag TEXT,  
    PRIMARY KEY(id_tweets, tag)  
);
```

1. Recall that certain constraints create indexes on the appropriate columns. List the equivalent `CREATE INDEX` commands that are run by these constraints.

**Note:** When solving all subsequent problems, assume only the indexes above have been created. For example, if problem 3 asks you to create an index, you should not assume that index has been created when working on problem 4.

2. List the scan methods applicable for the following SQL query.

```
SELECT count(*)
FROM users
WHERE id_users >= :min_id_users
      AND username >= :min_username
      AND username < :max_username
```

3. Create index(es) so that the following query can use an index only scan.  
Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT count(*)
FROM users
WHERE lower(username)=:username;
```

4. Create index(es) so that the following query can use an index only scan, avoid an explicit sort, and take advantage of the LIMIT clause for faster processing.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT id_users
FROM users
WHERE created_at <=: created_at
ORDER BY created_at DESC
LIMIT 10;
```

5. Create index(es) so that the following query will run as efficiently as possible.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT id_users, username
FROM users
JOIN tweets USING (id_users)
WHERE in_reply_to_user_id = :in_reply_to_user_id
LIMIT 10;
```

6. Create index(es) so that the following query will run as efficiently as possible.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT username, count(*)
FROM users
JOIN tweets USING (id_users)
WHERE in_reply_to_user_id = :in_reply_to_user_id
GROUP BY username;
```

7. Consider the following SQL query.

```
SELECT tag,count(*) AS count
FROM tweet_tags
GROUP BY tag
ORDER BY count DESC
LIMIT 10;
```

- a) Create index(es) so that the query will run as efficiently as possible.  
Do not create any unneeded indexes; if no new indexes are needed, say so.

- b) Given the indexes created above, is the LIMIT clause likely to significantly speed up the query? Explain why/why not.